



Sign Up

Sign In

 Search packages

Search

csv-parser TS

3.2.0 • Public • Published a year ago

 [Readme](#)

 [Code](#) Beta

 0 Dependencies

 1261 Dependents

 46 Versions

csv-parser

404 badge not found  codecov 97% install size 28.9 kB

Streaming CSV parser that aims for maximum speed as well as compatibility with the **csv-spectrum** CSV acid test suite.

`csv-parser` can convert CSV into JSON at a rate of around 90,000 rows per second. Performance varies with the data used; try `bin/bench.js <your file>` to benchmark your data.

`csv-parser` can be used in the browser with **browserify**.

neat-csv can be used if a `Promise` based interface to `csv-parser` is needed.

Note: This module requires Node v8.16.0 or higher.

Benchmarks

⚡ csv-parser is greased-lightning fast

→ npm run bench

Filename	Rows Parsed	Duration
backtick.csv	2	3.5ms
bad-data.csv	3	0.55ms
basic.csv	1	0.26ms
comma-in-quote.csv	1	0.29ms
comment.csv	2	0.40ms
empty-columns.csv	1	0.40ms
escape-quotes.csv	3	0.38ms
geojson.csv	3	0.46ms
large-dataset.csv	7268	73ms
newlines.csv	3	0.35ms
no-headers.csv	3	0.26ms
option-comment.csv	2	0.24ms
option-escape.csv	3	0.25ms
option-maxRowBytes.csv	4577	39ms
option-newline.csv	0	0.47ms
option-quote-escape.csv	3	0.33ms
option-quote-many.csv	3	0.38ms
option-quote.csv	2	0.22ms
quotes+newlines.csv	3	0.20ms
strict.csv	3	0.22ms
latin.csv	2	0.38ms
mac-newlines.csv	2	0.28ms
utf16-big.csv	2	0.33ms
utf16.csv	2	0.26ms
utf8.csv	2	0.24ms

Install

Using npm:

```
$ npm install csv-parser
```

Using yarn:

```
$ yarn add csv-parser
```

Usage

To use the module, create a readable stream to a desired CSV file, instantiate `csv`, and pipe the stream to `csv`.

Suppose you have a CSV file `data.csv` which contains the data:

```
NAME,AGE
Daffy Duck,24
Bugs Bunny,22
```

It could then be parsed, and results shown like so:

```
const csv = require('csv-parser')
const fs = require('fs')
const results = [];

fs.createReadStream('data.csv')
  .pipe(csv())
  .on('data', (data) => results.push(data))
  .on('end', () => {
    console.log(results);
    // [
```

```
// { NAME: 'Daffy Duck', AGE: '24' },  
// { NAME: 'Bugs Bunny', AGE: '22' }  
// ]  
});
```

To specify options for `csv` , pass an object argument to the function. For example:

```
csv({ separator: '\t' });
```

API

`csv([options | headers])`

Returns: `Array[Object]`

options

Type: `Object`

As an alternative to passing an `options` object, you may pass an `Array[String]` which specifies the headers to use. For example:

```
csv(['Name', 'Age']);
```

If you need to specify options *and* headers, please use the the object notation with the `headers` property as shown below.

escape

Type: `String`

Default: `"`

A single-character string used to specify the character used to escape strings in a CSV row.

headers

Type: `Array[String] | Boolean`

Specifies the headers to use. Headers define the property key for each value in a CSV row. If no `headers` option is provided, `csv-parser` will use the first line in a CSV file as the header specification.

If `false`, specifies that the first row in a data file does *not* contain headers, and instructs the parser to use the column index as the key for each column. Using `headers: false` with the same `data.csv` example from above would yield:

```
[
  { '0': 'Daffy Duck', '1': 24 },
  { '0': 'Bugs Bunny', '1': 22 }
]
```

Note: If using the `headers` for an operation on a file which contains headers on the first line, specify `skipLines: 1` to skip over the row, or the headers row will appear as normal row data. Alternatively, use the `mapHeaders` option to manipulate existing headers in that scenario.

mapHeaders

Type: `Function`

A function that can be used to modify the values of each header. Return a `String` to modify the header. Return `null` to remove the header, and it's column, from the results.

```
csv({
  mapHeaders: ({ header, index }) => header.toLowerCase()
})
```

Parameters

header *String* The current column header.

index *Number* The current column index.

mapValues

Type: `Function`

A function that can be used to modify the content of each column. The return value will replace the current column content.

```
csv({  
  mapValues: ({ header, index, value }) => value.toLowerCase()  
})
```

Parameters

header *String* The current column header.

index *Number* The current column index.

value *String* The current column value (or content).

newline

Type: `String`

Default: `\n`

Specifies a single-character string to denote the end of a line in a CSV file.

quote

Type: `String`

Default: `"`

Specifies a single-character string to denote a quoted string.

raw

Type: `Boolean`

If `true`, instructs the parser not to decode UTF-8 strings.

separator

Type: `String`

Default: `,`

Specifies a single-character string to use as the column separator for each row.

skipComments

Type: Boolean | String

Default: false

Instructs the parser to ignore lines which represent comments in a CSV file. Since there is no specification that dictates what a CSV comment looks like, comments should be considered non-standard. The "most common" character used to signify a comment in a CSV file is "#". If this option is set to true, lines which begin with # will be skipped. If a custom character is needed to denote a commented line, this option may be set to a string which represents the leading character(s) signifying a comment line.

skipLines

Type: Number

Default: 0

Specifies the number of lines at the beginning of a data file that the parser should skip over, prior to parsing headers.

maxRowBytes

Type: Number

Default: Number.MAX_SAFE_INTEGER

Maximum number of bytes per row. An error is thrown if a line exceeds this value. The default value is on 8 peta byte.

strict

Type: Boolean

Default: false

If true, instructs the parser that the number of columns in each row must match the number of headers specified or throws an exception. if false: the headers are mapped to the column index less columns: any missing column in the middle will result in a wrong property mapping! more columns: the additional columns will create a "_"+index properties - eg. "_10": "value"

outputByteOffset

Type: `Boolean`

Default: `false`

If `true`, instructs the parser to emit each row with a `byteOffset` property. The `byteOffset` represents the offset in bytes of the beginning of the parsed row in the original stream. Will change the output format of stream to be `{ byteOffset, row }`.

Events

The following events are emitted during parsing:

`data`

Emitted for each row of data parsed with the notable exception of the header row. Please see [Usage](#) for an example.

`headers`

Emitted after the header row is parsed. The first parameter of the event callback is an `Array[String]` containing the header names.

```
fs.createReadStream('data.csv')
  .pipe(csv())
  .on('headers', (headers) => {
    console.log(`First header: ${headers[0]}`)
  })
```

Readable Stream Events

Events available on Node built-in [Readable Streams](#) are also emitted. The `end` event should be used to detect the end of parsing.

CLI

This module also provides a CLI which will convert CSV to [newline-delimited](#) JSON. The following CLI flags can be used to control how input is parsed:

Usage: csv-parser [filename?] [options]

<code>--escape,-e</code>	Set the escape character (defaults to quote value)
<code>--headers,-h</code>	Explicitly specify csv headers as a comma separate
<code>--help</code>	Show this help
<code>--output,-o</code>	Set output file. Defaults to stdout
<code>--quote,-q</code>	Set the quote character ('"' by default)
<code>--remove</code>	Remove columns from output by header name
<code>--separator,-s</code>	Set the separator character ("," by default)
<code>--skipComments,-c</code>	Skip CSV comments that begin with '#'. Set a value
<code>--skipLines,-l</code>	Set the number of lines to skip to before parsing
<code>--strict</code>	Require column length match headers length
<code>--version,-v</code>	Print out the installed version

For example; to parse a TSV file:

```
cat data.tsv | csv-parser -s $'\t'
```

Encoding

Users may encounter issues with the encoding of a CSV file. Transcoding the source stream can be done neatly with a modules such as:

- `iconv-lite`
- `iconv`

Or native `iconv` if part of a pipeline.

Byte Order Marks

Some CSV files may be generated with, or contain a leading **Byte Order Mark**. This may cause issues parsing headers and/or data from your file. From Wikipedia:

The Unicode Standard permits the BOM in UTF-8, but does not require nor recommend its use. Byte order has no meaning in UTF-8.

To use this module with a file containing a BOM, please use a module like **strip-bom-stream** in your pipeline:

```
const fs = require('fs');

const csv = require('csv-parser');
const stripBom = require('strip-bom-stream');

fs.createReadStream('data.csv')
  .pipe(stripBom())
  .pipe(csv())
  ...
```

When using the CLI, the BOM can be removed by first running:

```
$ sed -i 's/\xEF\xBB\xBF//g' data.csv
```

Meta

CONTRIBUTING

LICENSE (MIT)

Keywords

csv parser fast json

Install

```
> npm i csv-parser
```

Repository

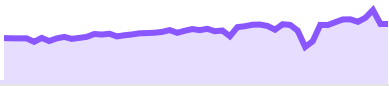
 github.com/mafintosh/csv-parser

Homepage

 github.com/mafintosh/csv-parser

Weekly Downloads

1,921,699

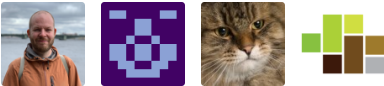


Version	License
3.2.0	MIT

Unpacked Size	Total Files
29.5 kB	6

Last publish
a year ago

Collaborators



 Analyze security with Socket

 Check bundle size

 [View package health](#)

 [Explore dependencies](#)

 [Report malware](#)



Support

[Help](#)

[Advisories](#)

[Status](#)

[Contact npm](#)

Company

[About](#)

[Blog](#)

[Press](#)

Terms & Policies

Policies

Terms of Use

Code of Conduct

Privacy

